

Web Programming Basics for Security

HTML & Forms

Web apps are built from HTML + forms + scripts:

```
<form action="/login" method="POST">
  <input name="user" type="text">
  <input name="pass" type="password">
  <button type="submit">Sign in</button>
</form>
```

The browser sends `user=alice&pass=secret` to `/login`. Attack surface: every input field is trust boundary.

HTTP Requests in Python

```
import requests

session = requests.Session()

# Login (URL-encode params)
r = session.post("http://testapp/login", data={"user": "alice", "pass": "wrong"})
print(r.status_code, r.headers.get("Set-Cookie"))
print(session.cookies.get_dict())

# Access an authenticated page
r2 = session.get("http://testapp/dashboard")
print(r2.text[:200])
```

A `requests session` persists cookies – just like a browser. This is how security testers script multi-step attacks.

Cookies & Sessions In Code

The server sets a session cookie; subsequent requests send it back:

```
# Manually send a stolen cookie
r = requests.get("http://testapp/private", headers={
    "Cookie": "session=IMPOSTER_VALUE"
})
```

The lesson: if the server trusts the cookie alone (no IP check, no MFA re-check), a stolen session cookie = full account takeover.

Enumerating Endpoints

```
import requests
for path in ["/admin", "/robots.txt", "/backup.zip", "/.git/config"]:
    r = requests.get("http://testapp" + path, timeout=3)
    print(f"{r.status_code:3d}  {path}" + ("  <-- interesting" if r.status_code in (200, 301, 401) else ""))
```

This endpoint enumeration is the heart of recon. You'll also do it with tools (gobuster, ffuf) – but now you understand what they do.

The DOM & JavaScript

The DOM is the browser's in-memory model of the page. JS runs client-side. **XSS** (cross-site scripting) happens when user input lands in the DOM without sanitization – letting an attacker run their JS in the victim's browser.

[← Python Fundamentals for Secu...](#)

[Writing Your Own Security Tool...](#)